



DEMO TASK — AI ENGINEER



Objective

Build a system that behaves like a **compiler for software generation**:

Natural language → structured config → validated → executable → produces a working application (via a runtime)

This is not a prompt engineering task.

This is a **system design + reliability + control problem**.

Reference - <https://base44.com/>

Submission Link - <https://forms.gle/ehWzSkSvhyXrGqk6>



Problem Statement

Users will input open-ended instructions like:

“Build a CRM with login, contacts, dashboard, role-based access, and premium plan with payments. Admins can see analytics.”

Your system must convert this into a **strict, complete, and reliable configuration** that includes:

- UI schema (pages, components, layouts)
 - API schema (endpoints, methods, validation)
 - Database schema (tables, relations)
 - Auth system (roles, permissions)
 - Business logic (e.g., premium gating, role access)
-



What You MUST Build

1. Multi-Stage Generation Pipeline (MANDATORY)

You MUST break the system into stages:

1. **Intent Extraction**
 - Parse user intent into structured intermediate form
2. **System Design Layer**
 - Convert intent → app architecture
 - Define entities, flows, roles
3. **Schema Generation**
 - Generate:
 - UI config
 - API config
 - DB schema
 - Auth rules
4. **Refinement Layer**
 - Resolve inconsistencies across layers

👉 Single prompt = immediate rejection

2. Strict Schema Enforcement

Define a **clear contract** for output.

Your system must guarantee:

- valid JSON (always)
- required fields present
- type safety
- cross-layer consistency

Example:

- API fields must match DB schema
 - UI fields must map to API
-

3. Validation + Repair Engine (CORE)

Your system must detect and handle:

- invalid JSON
- missing keys
- hallucinated fields
- schema mismatches
- logical inconsistencies

Then:

- repair automatically OR
- re-generate specific parts (not full retry blindly)

👉 This is the most important part of the task

4. Deterministic Behavior (HIGH BAR)

Your system should aim for:

Same input → consistent output (within reasonable variance)

Techniques may include:

- structured prompting
 - constrained decoding
 - modular generation
-

5. Execution Awareness (CRITICAL DIFFERENCE)

Your output must be:

directly usable to generate a working app (no manual fixes)

To prove this, you MUST:

- either:
 - integrate with a basic runtime (even minimal), OR
 - simulate execution and validate correctness

If your output cannot be executed → fail

6. Failure Handling System

Handle:

- vague prompts
- conflicting requirements
- underspecified inputs

Your system should:

- ask for clarification OR
 - make reasonable assumptions (and document them)
-

7. Evaluation Framework (SERIOUS SIGNAL)

Create a dataset:

- 10 real product prompts
- 10 edge cases:
 - vague
 - conflicting
 - incomplete

Track:

- success rate
- retries per request
- failure types
- latency

👉 Show actual metrics, not claims

8. Cost vs Quality Tradeoff (ADVANCED)

Demonstrate:

- how you balance:
 - latency
 - cost
 - output quality

Even basic analysis = strong signal



What We Will Test

We will:

1. Give completely new prompts

2. Modify requirements mid-way
3. Introduce ambiguity

We will check:

- Is output usable?
- Is it consistent?
- Does it break under pressure?

If yes → reject



Submission Requirements (STRICT)

Submit via Google Form:

1. **Live URL (preferred, strong signal)**
 - Host a simple interface where:
 - we can enter a prompt
 - see generated JSON output
-

2. GitHub Repository

- Clean, well-structured code
 - Clear pipeline separation
-

3. Loom Video (5- 10 minutes)

Explain:

- architecture (end-to-end)
 - pipeline design (why multi-step)
 - validation + repair system
 - how you ensure reliability
 - tradeoffs (quality vs latency vs cost)
-



Evaluation Criteria (HIGH BAR)

We are not just evaluating

- prompt tricks
- UI
- surface-level output

But We ARE evaluating more on :

1. System Thinking

- does this feel like an engineered system or a script?

2. Reliability

- does it handle real-world messiness?

3. Control Over LLMs

- are outputs predictable and structured?

4. Execution Awareness

- can this actually power a product?

5. Depth of Thinking

- tradeoffs, constraints, decisions

Exceptional (Top 1%)

- modular pipeline (like a compiler)
- intelligent repair (not brute retry)
- strong consistency
- clear evaluation metrics
- output works with runtime system

Final Note

This is not a tutorial task.

You are expected to:

- design systems, not scripts
- handle ambiguity
- make tradeoffs
- build something that could be used in production

Use AI tools, but you must understand what you build.

Here's a clean, copy-paste-ready version you can include in the task:

Important Note

This task is **intentionally challenging**.

We have designed it to create a **steep learning curve** and to identify people who can operate with **high ownership and high agency**.

You are not expected to know everything beforehand.

But you are expected to:

- figure things out independently
- navigate ambiguity
- make decisions without waiting for instructions
- push through technical and product challenges

This is **not a typical internship assignment**.

It reflects the kind of problems you will work on here.

If you're someone who enjoys:

- building from scratch
- solving unclear problems
- taking full ownership

you will find this exciting.

If you prefer structured, step-by-step tasks, this may not be the right fit